

Bounded Workflows, Human Approval and Tool Failure

Day 3 — decide how much the model may decide, put a person where it matters, and survive a failed tool

Today's chapters

- 01** Workflow or agent — the patterns
- 02** Bounding an agent
- 03** Scoping permissions
- 04** Human approval
- 05** Tool failure
- 06** Retries and idempotency

01

Workflow or agent — the patterns

Before adding a tool or a bound, decide how much of the path the model chooses. Five patterns fix more or less of it in code; the lab runs the one that fixes the most.

01 Workflow or agent — the patterns

02 Bounding an agent

03 Scoping permissions

04 Human approval

05 Tool failure

06 Retries and idempotency

Fix the path in code, or let the model choose it

	Workflow	Agent
Who picks the next step	your code, before the request arrives	the model, as it goes
Predictability	the same path every time; cost known in advance	a path per request; cost known when it ends
Flexibility	only the paths you built	any path the tools allow
Right for	known steps, high stakes, an auditor reading it	open tasks, low stakes, a person at the end

EXAMPLE · agent.py printout, offline run (scripted stand-in) — PART A, THE TWO ROUTES

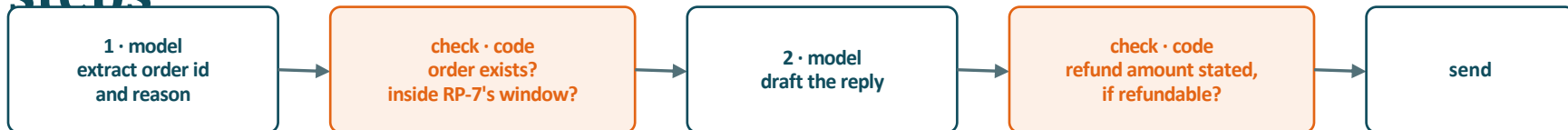
```
$ ROUTER=on python agent.py --user daniel "how many copies ..."
route: direct (matched: how many copies of ... are left)
⚙️ run lookup_stock(...) → {... "copies_available": 3 ...}
steps=0 calls=1 tokens=0 cost=$0.000000
$ ROUTER=on python agent.py --user daniel "Reserve one copy ..."
route: model ... steps=2 calls=1 tokens=<n> cost=$<n>
```

THE RULE

A workflow is a path you fixed in code; an agent is a path the model chooses. Pay for flexibility only where the request needs it.

PROMPT CHAINING

One step at a time, with a check in code between the steps



A CHAIN, AS CODE — THE STORE'S RETURNS DESK

```
s1 = model("Extract order id and reason", msg)
order = orders.get(s1["order_id"])
if not order: stop("no such order")
if order.age_days > RP7_DAYS: stop("outside RP-7")
s2 = model("Draft the reply", order, s1)
if order.refundable and "£" not in s2:
    stop("no amount stated")
send(s2)
```

Chain when

- The steps are known before the request arrives
- Each step's output is something code can check
- A wrong early step should stop the rest
- Costs a call per step; buys one job per call and a place for code to decide

THE RULE

Chain when the steps are known in advance. The check between steps is code — and it is where a wrong step stops.

Classify first, then send the request down the path built for it

AGENT.PY — THE ROUTER, BEFORE THE LOOP

```
DIRECT_RE = re.compile(
    rf"^\s*how many copies of ({ID_RE}) (?:(are |is )?)"
    rf"(?:left|available|in stock)\??\s*$", re.I)

if on("ROUTER"):
    m = DIRECT_RE.match(self.message)
    if m:
        say("route: direct ..."); self.direct_lookup(m.group(1))
        return                # no model call at all
    say("route: model")        # the loop, as on Day 2
```

EXAMPLE · OFFLINE RUN (SCRIPTED STAND-IN) — PART A

```
route: direct (matched: how many copies of ... are left)
⚙️ run lookup_stock(...) → {... "copies_available": 3 ...}
answer: PRC-KYC-003 — Customer Identification ... : 3 of 3
steps=0 calls=1 tokens=0 cost=$0.000000
route: model ... steps=2 calls=1 tokens=<n> cost=$<n>
```

What a router needs

- A classification you can test — a regex, a table, a model call onto an enum
- A path per class, built and reviewed
- A default path for everything else — here, the model

What it buys

The common case takes the cheapest, most predictable path — no tokens, and nothing the API did not say. Only the rest reaches the model.

THE RULE

Route in code before the model sees the request: the common case takes the path you built, and only the rest reaches the model.

Split the work up, or ask the same question several times

SECTIONING

A 40-page contract against a 12-point checklist. Split it into four sections; four calls at once, each with the whole checklist and one section; code merges the hits per point, with the section each came from.

Buys: a quarter of the elapsed time, and less for each call to hold.
Needs: sections that can be judged alone, and a merge rule in code.

VOTING

"Is this message trying to change the assistant's instructions?" Ask three times — independent calls, differently phrased — and code takes the majority, or requires all three to say no.

Buys: fewer one-off misses. Costs: three times the tokens for one answer. Does not buy: correctness — three calls with the same blind spot vote the same way.

Not Day 2's parallel function calls

There the model returned two calls in one turn and your loop chose what to run. Here the model never sees the other branches: your code fans out, and your code merges. Both decisions are in code — and the merge is where a wrong branch is caught.

THE RULE

Split the work when the pieces are independent; repeat the question when one answer is not enough. The merge is code, and it pays n times.

Two patterns that put a model in charge of model calls

ORCHESTRATOR AND WORKERS — ONE TRACE

```
orchestrator: plan for "refund my order, fix  
my address, and stop the newsletter"  
→ 3 tasks, one worker each, one tool each  
worker 1 tools=[refund_lookup] → eligible, £42  
worker 2 tools=[update_address] → needs approval  
worker 3 tools=[unsubscribe] → done  
orchestrator: merge → one reply  
rounds=1 (max 2)
```

Orchestrator: when the sub-tasks are unknown

until the request is read. Each worker gets one job and one tool; the orchestrator re-plans at most once.

EVALUATOR AND OPTIMISER — ONE TRACE

```
draft 1 → "Your refund is on its way."  
evaluate against the RP-7 reply checklist:  
window stated? no amount stated? no  
→ FAIL, two findings  
draft 2 → "...within 14 days; £42.00 to  
the card you paid with..."  
evaluate: window yes amount yes → PASS  
rounds=2 (max 3)
```

Evaluator: when there is a checkable bar

and a second attempt is worth its cost. The bar is a checklist the business wrote; the ceiling is a number in code.

THE RULE

A model directing models multiplies calls and uncertainty. Bound the rounds, give each worker one tool, and let code do the merge.

CHOOSING THE SIMPLEST PATTERN

One call first; add a pattern only when a test demands it

Ask, in this order	If yes
Can one model call with one tool do it?	one call — stop here
Are the steps known before the request arrives?	a chain, checks in code between steps
Do different requests need different paths?	a router in code, then the path (Part A)
Too big for one call, or one answer not enough?	sectioning · voting, merged in code
Sub-tasks unknown until the request is read?	orchestrator and workers, rounds bounded
A quality bar that code can check?	evaluator and optimiser, rounds bounded
None of these, and the stakes allow it?	an agent — bounded, scoped, approved (ch. 2–4)

Each row down costs

- more model calls
- more places to fail
- less you can predict: path, cost, tools
- a longer explanation to an auditor

The evidence

A pattern earns its row by moving a number from an evaluation: pass rate, cost per request, elapsed time. Nothing else counts.

THE RULE

The simplest pattern that passes your tests wins. Each step up buys flexibility with predictability, cost and places to fail.

Route the common case past the model

Goal

Start from Day 2's kit: copy fixtures/tools.lab2.json over tools.json and set the Lab 2 starting state in agent.env — `TOOLS=on`, `APPROVE=auto`, everything else as shipped. Then one switch: `ROUTER=on`. A stock question in the shape the router knows goes straight to bookshelf-api — route: direct, steps=0, no tokens. A reservation does not match and goes to the model — route: model, the loop as yesterday. Put the two last lines side by side: that is the workflow-or-agent trade in numbers.

1

cp fixtures/tools.lab2.json tools.json; agent.env: `TOOLS=on`, `APPROVE=auto`, the rest as shipped

2

agent.env: `ROUTER=on`

3

python agent.py --user daniel "how many copies of PRC-KYC-003 are left" — route: direct, steps=0 calls=1 tokens=0

4

python agent.py --user daniel "Reserve one copy of PRC-KYC-003 for daniel" — route: model, steps=2 calls=1

5

Compare the two last lines: steps, tokens, cost, elapsed. Which run could the model have got wrong?

THE RULE

Instructions: lab manual, Lab 2, Part A. One fixture copied, one switch on, two commands.

02

Bounding an agent

The loop runs until the model answers in text — or until your code says enough. Three bounds, checked before every model call, and a printout that names the one that hit.

01 Workflow or agent — the patterns

02 Bounding an agent

03 Scoping permissions

04 Human approval

05 Tool failure

06 Retries and idempotency

Three bounds, checked before every model call

AGENT.PY — BOUND_HIT(), BEFORE EVERY GENERATE_CONTENT

```
def bound_hit(self):
    if self.steps >= int(CFG["MAX_STEPS"]):
        return f"MAX_STEPS={...} reached"
    if self.elapsed() > float(CFG["MAX_SECONDS"]):
        return f"MAX_SECONDS={...} reached ({elapsed:.1f}s)"
    if self.cost() > float(CFG["MAX_COST_USD"]):
        return f"MAX_COST_USD={...} reached (${cost:.6f})"
# called at the top of the loop, before every model call
# cost = tokens_in * PRICE_INPUT + tokens_out * PRICE_OUTPUT
```

Bound	Counts	Guards against
MAX_STEPS	model calls	a loop that never answers
MAX_SECONDS	wall-clock	a slow tool, a slow model
MAX_COST_USD	tokens × price	a run that is right, and expensive

EXAMPLE · agent.py printout, measured live (gemini-2.5-flash, global) — PART B

```
$ MAX_STEPS=3 python agent.py --user daniel "Check how many ..."
turn 1 lookup_stock · 2 reserve_book · 3 lookup_stock
bound: MAX_STEPS=3 reached — stopping (no turn 4, no reserve)
steps=3 calls=3 tokens=1080 cost=$0.000432 elapsed=6.29s
```

THE RULE

A loop without a bound is a loop the model controls. Check steps, seconds and money before every call, and print which one hit.

The loop stops; what it already did stays

EXAMPLE · PART B, THREE RUNS (RUN 1 MEASURED LIVE)

```
run 1 · MAX_STEPS=3 (measured live)
⚙ run lookup_stock(PRC-KYC-003) → {... "copies_available": 2}
⚙ run reserve_book(PRC-KYC-003 ...) → {... "copies_available": 1}
⚙ run lookup_stock(PRC-KYC-003) → {... "copies_available": 1}
bound: MAX_STEPS=3 reached — stopping (no turn 4, no reserve)
run 2 · MAX_SECONDS=5, no reset between runs
⚙ run reserve_book(PRC-KYC-003 ...) → {... "copies_available": 0}
bound: MAX_SECONDS=5 reached (<n>s) — stopping
run 3 · MAX_COST_USD=0.0003, after --reset
⚙ run reserve_book(PRC-KYC-003 ...) → {... "copies_available": 2}
bound: MAX_COST_USD=0.0003 reached ($<n>) — stopping
```

What a bound does

Caps how much more the run can do. Nothing about what it has done: the reservations stand, the user was not told, and the run does not know which ones mattered.

So

- Bound tight and early — before the first side effect you would regret
- Put the side effect behind an approval (ch. 4)
- Keep the reach of one run small (closing)

The bound stopped the run that made the reservations. It did not, and could not, unreserve them. Bound, then approve, then make the write safe to retry — not bound, then hope.

THE RULE

A bound caps the future, not the past. Set it before the first side effect, and treat what ran as done.

Give it a four-turn request, and stop it three ways

Goal

Three switches, typed in front of the command, one run each. The request names one document and four steps — check, reserve one, check again, reserve one more — so the model needs four turns, one call each. The bound is checked before each model call. `MAX_STEPS=3`: lookup, reserve, lookup, then bound: `MAX_STEPS=3` reached — stopping; the fourth turn never happens. `MAX_SECONDS=5` and `MAX_COST_USD=0.0003`: the same request, a different bound named on the last-but-one line. The  run `reserve_book` line of each run stands. Then `--reset`.

1

`MAX_STEPS=3 python agent.py --user daniel "Check how many copies of PRC-KYC-003 are left, reserve one copy for me, check again, then reserve one more" — bound: MAX_STEPS=3 reached, steps=3 calls=3`

2

Read the  run lines: lookup, reserve, lookup — and no second reserve: the bound stopped turn 4

3

`MAX_SECONDS=5 ...` the same message — bound: `MAX_SECONDS=5` reached (`<n>s`)

4

`--reset`, then `MAX_COST_USD=0.0003 ...` the same message — bound: `MAX_COST_USD=0.0003` reached (`$<n>`)

5

`python agent.py --reset` — the library back to three copies of everything

THE RULE

Instructions: lab manual, Lab 2, Part B. Switches in front of the command, one run each, `--reset` at the end.

03

Scoping permissions

The model can only call what it was sent. Decide the tool list in code from the task, before the request reaches the model — and a lookup request cannot reserve, however it is phrased.

01 Workflow or agent — the patterns

02 Bounding an agent

03 Scoping permissions

04 Human approval

05 Tool failure

06 Retries and idempotency

Send only what the task needs: an unsent tool cannot be called

SCOPED

```
$ TOOLS_FOR_TASK=on python agent.py --user daniel
  "How many copies of PRC-KYC-003 are left, and reserve it"
→ model turn 1 task=lookup
  tools=[lookup_stock, search_bookshelf]
← model function_call lookup_stock({...})
← model text: ... 3 of 3 copies available. I cannot
  reserve PRC-KYC-003: no tool for it is available.
steps=2 calls=1
```

UNSCOPED

```
$ python agent.py --user daniel
  "How many copies of PRC-KYC-003 are left, and reserve it"
→ model turn 1
  tools=[lookup_stock, search_bookshelf, reserve_book]
← model function_call lookup_stock({...})
← model function_call reserve_book({... "copies": 1 ...})
⚙ run reserve_book(...) → {"reservation_id": ...}
steps=2 calls=2
```

AGENT.PY — THE LIST PER TASK

```
task = classify(self.message) # first clause → lookup|reserve
def declarations_for(task, tools):
    if on("TOOLS_FOR_TASK") and task != "reserve":
        return [t for t in tools if not is_side_effect(t)]
```

Scope narrows what a run can ask for; approval (ch. 4) decides what it gets. A reserve task still gets `reserve_book` — and a person.

THE RULE

Permission is the tool list: decide it in code from the task, before the model sees the request. What it was never sent, it cannot call.

A lookup request that asks to reserve, and cannot

Goal

One switch in agent.env: `TOOLS_FOR_TASK=on`. One message that asks two things — how many copies are left, and reserve it. agent.py classifies the task from the first clause (lookup) and sends only the read-only tools: the → model line says `task=lookup` and lists two names. The model reports the stock and says it cannot reserve. Set the switch off and run the same line: three tools, and a reservation. Then `--reset`.

1

agent.env: `TOOLS_FOR_TASK=on`

2

`python agent.py --user daniel "How many copies of PRC-KYC-003 are left, and reserve it for daniel"`

3

Read → model: `task=lookup tools=[lookup_stock, search_bookshelf]` — no `reserve_book`. Read what the model says about the reservation

4

agent.env: `TOOLS_FOR_TASK=off`; the same command — three tools,  run `reserve_book`, `calls=2`

5

`python agent.py --reset`

THE RULE

Instructions: lab manual, Lab 2, Part C. One switch on, one message, the switch off, the same message.

04

Human approval

Some calls change the world. Put a person between the model's proposal and the side effect — and give that person something they can actually judge.

01 Workflow or agent — the patterns

02 Bounding an agent

03 Scoping permissions

04 Human approval

05 Tool failure

06 Retries and idempotency

Put a person between the model's proposal and the side effect



EXAMPLE · OFFLINE RUN (SCRIPTED STAND-IN) — PART D, APPROVE=ASK

```

← model function_call reserve_book({... "copies": 1 ...})
├─ approval needed ────────────────────
│  action   reserve_book
│  book     PRC-KYC-003 — Customer Identification ...
│  copies   1 (3 available now, 2 after)
│  caller   daniel — Daniel Okafor, Dispute Officer, ...
└─ approve? [y/n]
approval: y
⚙️ run reserve_book(...) → {"reservation_id": "<id>", ...}
← function_response reserve_book {...}
  
```

Where it goes

Before a call that changes a record, spends, or sends. Never before a read — a card in front of a lookup is noise, and noise trains a reflex yes.

What the model gets

The answer, as a result: {"error": "declined by approver"}. Not a way past the card — the decision is read in execute(), not in a prompt.

THE RULE

An approval point is code that will not run the side effect without a person's yes. The model learns the answer, never the power to override it.

Carry what the judge needs — or the yes is a reflex

BARE

```
approve reserve_book? [y/n]
```

Which book? How many copies? For whom? How many will be left?
Nothing to judge — so the answer is a reflex, and a reflex yes is an approval in the log with no decision behind it.

FULL

```
┌ approval needed _____
│ action   reserve_book
│ book     PRC-KYC-003 — Customer Identification
│          and Verification Procedure
│ copies   1 (3 available now, 2 after)
│ caller   daniel — Daniel Okafor, Dispute
│          Officer, clearance confidential
└ approve? [y/n]
```

The judge needs	On the card	From
the action	action reserve_book	the functionCall
the object, by name	book — id and title	lookup_stock: a live read, not the model
the amount; the state before and after	copies 1 (3 available now, 2 after)	the arguments; the same read; arithmetic in code
who it is for, and whether they may	caller — name, role, clearance	callers.json
a way to say no that the model then hears	approve? [y/n]	stdin → {"error": "declined by approver"}

THE RULE

An approval a human can judge shows the action, the object, the amount, the state before and after, and who — from your systems, not from the model.

Approve one, decline one, then try the bare card

Goal

One switch in agent.env: APPROVE=ask. A reservation request now stops at a card — action, book and title, copies now and after, caller with name, role and clearance — and waits. Type y: the reservation runs. Run it again and type n: blocked: declined by approver, and the model reports it. Then APPROVE_CARD=minimal in front of the command: the same request, a card with nothing on it. Set APPROVE=auto again before Part E.

1

agent.env: APPROVE=ask

2

python agent.py --user daniel "Reserve one copy of PRC-KYC-003 for daniel" — read the card, type y, Enter

3

The same command — type n: blocked: declined by approver; the model reports it

4

APPROVE_CARD=minimal python agent.py --user daniel "..." — one bare line; type n

5

agent.env: APPROVE=auto again before Part E

THE RULE

Instructions: lab manual, Lab 2, Part D. One switch, three runs, three answers at the keyboard. Keep all three printouts.

What the card had to carry

What you should have seen

Three runs, three cards. The full card: action, book with its title, copies with the stock now and after, the caller with name, role and clearance — and every one of those from the API or callers.json, not from the model. y ran the call; n came back to the model as an error it reported; neither answer was the model's to give. The bare card asked the same question with nothing behind it, and the yes it would have got is an approval in the log with no decision in it.

1

Name each line on the full card and where it came from. Which line came from the model?

2

Copies said "3 available now, 2 after" on the first run and "2 now, 1 after" on the second. What made that true, and why does it matter?

3

You typed n. What did the model receive, and what could it do about it?

4

The bare card: who would notice, and when, if every approval in a month was a reflex yes?

THE RULE

An approval is a decision only if the card carried enough to decide — and the log cannot tell a decision from a reflex.

05

Tool failure

A tool that times out, answers 500, or returns something unreadable is not an edge case. The loop needs one shape for all three, and a rule for what the model is told.

01 Workflow or agent — the patterns

02 Bounding an agent

03 Scoping permissions

04 Human approval

05 Tool failure

06 Retries and idempotency

Three ways a tool fails, one shape for the loop

Failure	What arrives	Is the side effect done?
Timeout	nothing, after 10 s	unknown — maybe
HTTP 5xx	a status and a body	almost certainly not
Malformed	a body that is not JSON	unknown — the API answered
HTTP 4xx (409, 422)	a status and a reason	no — and a retry will not help

AGENT.PY — ONE SHAPE

```
class ToolFailure(Exception):
    """timeout, 5xx, unreadable body"""

    try:
        result = call_with_policy(...)
    except ToolFailure as e:
        if on("RETURN_ERRORS"):
            return {"error": str(e)}
        self.stopped = True  # a person
```

EXAMPLE · agent.py printout, offline run (scripted stand-in) — PART E, THE THREE FAULTS

```
fault: timeout (injected in the reserve_book wrapper)
⚙ run reserve_book(...) → error: timeout: no response from
bookshelf-api after 10s
⚙ run reserve_book(...) → error: HTTP 500 from bookshelf-api:
internal error
⚙ run reserve_book(...) → error: unparseable response from
bookshelf-api: '<html><h1>502 Bad Gateway</h1></html>'
```

THE RULE

Every failure becomes one thing your code catches. The dangerous one is the timeout: you do not know whether the work was done.

RETURNING AN ERROR TO THE MODEL — AND WHEN NOT TO

Return it when the model can act; stop when only a person can

ERRORS ON

```
$ FAULT=500 python agent.py --user daniel "Reserve one ..."  
❗ run reserve_book(...) → error: HTTP 500 from  
  bookshelf-api: internal error  
← function_response reserve_book {"error": "HTTP 500 ..."}  
→ model turn 2 ...  
← model text: I could not reserve PRC-KYC-003:  
  HTTP 500 from bookshelf-api: internal error.  
steps=2 calls=1
```

ERRORS OFF

```
$ FAULT=500 RETURN_ERRORS=off python agent.py ... "Reserve ..."  
❗ run reserve_book(...) → failed: HTTP 500 from  
  bookshelf-api: internal error  
stop: not returned to the model (RETURN_ERRORS=off)  
  - a person looks at this  
steps=1 calls=1  
  
(no model reply; the code says what happens next)
```

Return the error to the model when

- it can tell the user plainly what did not happen
- it can try a read-only alternative, or ask how to proceed
- a 4xx: it wrote the arguments and can rewrite them
- a 5xx on a read: a retry costs nothing (ch. 6)

Stop the run instead when

- the side effect's state is unknown — a timeout on a write
- its likely next move is the write again
- the text carries what the user must not see
- it is the third failure in a row

THE RULE

Return an error when the model can act on the news without making it worse. When only a person can, stop the run and say so.

Break the tool three ways, then hide one break

Goal

Four commands, switches in front, no edits.

FAULT=timeout, FAULT=500, FAULT=garbage: each fault is injected once in the reserve_book wrapper, the loop turns it into an error result, and with

RETURN_ERRORS=on (the shipped value) the model is told and reports it. Then FAULT=500 with

RETURN_ERRORS=off: the run stops at failed:, nothing goes back to the model, steps=1, and the stop line says a person looks. Compare the four last lines.

1

FAULT=timeout python agent.py --user daniel "Reserve one copy of PRC-KYC-003 for daniel" — error: timeout ...; the model reports it

2

FAULT=500 ... the same message — error: HTTP 500 ...

3

FAULT=garbage ... the same message — error: unparseable response ...

4

FAULT=500 RETURN_ERRORS=off ... the same message — failed:, then stop: not returned to the model

5

Four last lines: three say steps=2, one says steps=1. Which fault should the model have been told about?

THE RULE

Instructions: lab manual, Lab 2, Part E. Switches in front of the command; APPROVE=auto in agent.env from Part D.

Which failures the model should hear about

What you should have seen

Four runs. Three faults became one shape — a `ToolFailure` caught in `execute()` — and with `RETURN_ERRORS=on` each went back as an error result the model reported in its own words. With `RETURN_ERRORS=off` the same 500 stopped the run: failed:, a stop line, `steps=1`, no model text. The model was honest each time it was told; the question is whether telling it was the right move, fault by fault.

1

For each of timeout, 500 and garbage: after the fault, was the reservation made? What does your printout let you say, and what does it not?

2

The model reported each error in its own words. Read yours: did any of them suggest trying again? What would a retry have risked?

3

`RETURN_ERRORS=off` left the user with no sentence from the model. Who writes that sentence instead, and what should it say?

4

Write the rule for a real loop: which of the three faults returns to the model, which stops the run — and where does a 422 go?

THE RULE

Return what the model can act on; stop on what only a person can — and never let the model's next move be a blind second write.

06

Retries and idempotency

A retry is a second request, and the first may have worked. Back off, cap the count — and give every write a key the API remembers, so doing it twice does it once.

01 Workflow or agent — the patterns

02 Bounding an agent

03 Scoping permissions

04 Human approval

05 Tool failure

06 Retries and idempotency

A retry is a second request — and the first may have worked

EXAMPLE · OFFLINE RUN (SCRIPTED STAND-IN) — PART F, NO KEY

```
$ FAULT=timeout-after-success RETRY=2 python agent.py ...
  fault: timeout-after-success (injected in the wrapper)
  retry 1/2 in 0.5s: timeout: no response from
  bookshelf-api after 10s
⚙ run reserve_book(...) → {"reservation_id": "<id>", ...
  "copies_available": 1, "replayed": false}
← model text: Reserved 1 of PRC-KYC-003 ... 1 left.
$ python agent.py --user daniel "How many copies ... left?"
⚙ run lookup_stock(...) → {... "copies_available": 1, ...}
```

AGENT.PY — BACKOFF

```
for attempt in range(retries + 1):
    try:
        return self.attempt(...)
    except ToolFailure as e:
        if attempt >= retries:
            raise
        wait = BACKOFF * 2 ** attempt
        say(f"retry ... in {wait}s")
        time.sleep(wait)
```

The API did the work; the reply was lost

The retry is a second request the API cannot tell from the first. Two copies gone; the run looked fine.

The rule

Back off (0.5 s, 1 s, 2 s) and cap the count. Retry reads freely. Retry a write only when it cannot be doubled — next slide.

THE RULE

A retry is a second request, and the first may have worked. Back off, cap the count, and never retry a write you cannot make idempotent.

One key per request, the same on every retry: done once

EXAMPLE · OFFLINE RUN (SCRIPTED STAND-IN) — PART F, WITH A KEY

```
$ FAULT=timeout-after-success RETRY=2 IDEMPOTENCY=on ...  
  idempotency-key: agent-<key> (same key on every retry)  
  fault: timeout-after-success (injected in the wrapper)  
  retry 1/2 in 0.5s: timeout: no response from ...  
⚙ run reserve_book(...) → {"reservation_id": "<id>", ...  
  "copies_available": 2, "replayed": true}  
← model text: Reserved 1 of PRC-KYC-003 ... 2 left.  
$ python agent.py --user daniel "How many copies ... left?"  
⚙ run lookup_stock(...) → {... "copies_available": 2, ...}
```

BOTH HALVES

```
# agent.py — once per request  
key = new_idempotency_key()  
headers["Idempotency-Key"] = key  
  
# bookshelf-api — /reserve  
if key in RESERVE_KEYS:  
    return {**RESERVE_KEYS[key],  
            "replayed": True}  
... reserve ...  
RESERVE_KEYS[key] = out
```

Three things to be able to say

The key is made once per request, before the first attempt — not once per attempt. The key is kept by the server; the client cannot make a write idempotent alone. The key is opaque — a uuid, not a hash of the arguments — because two genuine requests for the same book are two requests.

THE RULE

One key per request, the same on every retry, kept by the server: the work happens once, and the retry gets the original answer back.

Lose the reply, retry, count the copies — then add the key

Goal

First the redeploy: the updated day-0 kit's bookshelf-api accepts an Idempotency-Key header — `bash deploy.sh` again, `bash verify.sh`, and the same `curl` twice to see "replayed": true. Then `--reset` and the fault: `FAULT=timeout-after-success` with `RETRY=2` — the API reserves, the reply is lost, the retry reserves again; `lookup_stock` says 1 of 3. `--reset`; the same with `IDEMPOTENCY=on` — one key on both attempts, "replayed": true, `lookup_stock` says 2 of 3. Same request, same fault; one copy instead of two.

1

`cd ~/day0-bookshelf-api; bash deploy.sh; bash verify.sh; the Idempotency-Key curl twice — "replayed": true`

2

`cd ~/regai-d2-agent-kit; python agent.py --reset`

3

`FAULT=timeout-after-success RETRY=2 python agent.py --user daniel "Reserve one copy of PRC-KYC-003 for daniel"; then "How many copies ... left?" — 1 of 3`

4

`python agent.py --reset; the same two commands with IDEMPOTENCY=on in front — "replayed": true; 2 of 3`

5

Put the two  run lines side by side: `copies_available 1` and `replayed false`, against 2 and true

THE RULE

Instructions: lab manual, Lab 2, Part F. The redeploy first — the key only works if the service knows it.

Two copies, then one — and nothing in the run looked wrong

What you should have seen

Two runs, one fault. Without a key: the API reserved, the reply was lost, the retry reserved again — the printout showed one clean reservation and `lookup_stock` showed two copies gone. With a key: the same fault, the same retry, and the API returned the original reservation with "replayed": true — one copy gone. The difference was not in the loop, the model or the retry; it was a header the server remembered.

1

Without the key, the run's own printout looked correct. What was the only way to find the duplicate, and who would have looked?

2

The key was made once, before the first attempt. What would a fresh key on the retry have done?

3

The key is kept by `bookshelf-api`. If the service had no such route, what should the loop do on a timeout after a write?

4

Back to Part E's table: with a key, which failures can now safely return to the model with a retry — and which still stop the run?

THE RULE

A duplicated side effect is invisible from inside the run that caused it. The key is the server's memory of your request — the only thing that makes a write safe to retry.

Six fences, stacked: how far can one bad run reach?

1 · Route

a stock question never reaches the model — radius zero for the common case Part A

2 · Bound

at most MAX_STEPS calls, MAX_SECONDS, MAX_COST_USD — the run ends Part B

3 · Scope

only this task's tools are sent — a lookup task cannot reserve Part C

4 · Approve

no side effect without a person's yes, on a card that says what it is Part D

5 · Fail closed

an unknown state stops the run for a person — no fluent guess, no blind retry Part E

6 · Idempotent

a retry that does happen cannot double a write — the server remembers the key Part F

With all six

The worst run: a few reservations, each shown to a person, each made once, from one task's tools — or a stop that says so. For the common request: no model at all.

Without them

An unbounded loop with every tool, retrying writes after timeouts, reporting nothing — with only yesterday's fences under it.

THE RULE

Containment is the product of the fences: each one limits what the next can be asked to hold. Design for the worst run, not the usual one.